

Understanding and Implementing CSRF Tokens in Express.js

1. Introduction to CSRF

1.1 What is CSRF?

CSRF (Cross-Site Request Forgery) is an attack in which:

- A malicious user creates a fake HTML form on another website.
- This form's **action** attribute is set to your website's route.
- When victims unknowingly submit the malicious form, harmful requests are sent to your server.
- Hackers may repeatedly submit fake forms, filling your database and overloading your server.

1.2 Why Does CSRF Happen?

HTML forms reveal:

- Form **action** URLs
- Input field names Because hackers can read and replicate these, they can forge requests unless the requests are validated.

1.3 How CSRF Tokens Prevent Attacks

A CSRF protection flow in Express.js:

1. User requests a form page.
2. Server generates a unique, random token.
3. Server stores token in the session or cookies.
4. Server embeds token into the form as a **hidden input**.
5. On form submission:
 - Server compares the submitted token with the stored token.
 - If tokens match → request is valid.
 - If tokens do NOT match → submission is rejected.

2. Required Packages

To implement CSRF protection in Express, two packages are required:

2.1 **cookie-parser**

Enables Express to read and set cookies.

2.2 csurf

Automatically:

- Generates CSRF tokens
- Validates submitted tokens

3. Installation Commands

Install both packages:

```
npm install cookie-parser csurf
```

If `csurf` becomes deprecated in the future, a maintained alternative is:

```
npm install @akshaykrit.com/csrf
```

4. Setting Up CSRF Protection in Express.js

4.1 Import Modules

```
import express from "express";
import cookieParser from "cookie-parser";
import csrf from "csurf";

const app = express();
```

4.2 Configure Middleware

Enable cookie parsing

```
app.use(cookieParser());
```

Create CSRF protection middleware

```
const csrfProtection = csrf({ cookie: true });
```

5. Using CSRF Protection in Routes

5.1 Apply CSRF Middleware to Form Route

When rendering the form:

```
app.get("/myform", csrfProtection, (req, res) => {
  res.render("myform.twig", {
    csrfToken: req.csrfToken()
  });
});
```

5.2 Embed Token Inside the Form (Using Twig)

Inside `myform.twig`:

```
<form action="/submit" method="POST">
  <input type="hidden" name="_csrf" value="{{ csrfToken }}">

  <label>Name:</label>
  <input type="text" name="name">

  <button type="submit">Submit</button>
</form>
```

Note: The `name="_csrf"` field **must** match this exact name. The `csurf` package expects this field.

6. Processing Form Submission with CSRF Validation

Create the POST route:

```
app.post("/submit", csrfProtection, (req, res) => {
  // If token is invalid, csurf automatically throws an error
  res.send(req.body);
});
```

If the token is invalid or missing:

- Server responds with **403 Forbidden**
 - Error message: *Invalid CSRF Token*
-

7. Verifying Token Behavior

7.1 Token in Page Source

When viewing the source, you should see:

```
<input type="hidden" name="_csrf" value="random-generated-token">
```

Each refresh generates a new random token.

7.2 Successful Submission

If the token matches:

- The form data is shown (e.g., { name: "John", _csrf: "value..." })

7.3 Failed Submission

If the token:

- Has extra spaces
- Is modified
- Is missing Then server returns:

```
ForbiddenError: invalid csrf token
```

This protects the application from CSRF attacks.

8. Summary of CSRF Workflow

1. Install `cookie-parser` and `csrf`
2. Enable cookie parsing middleware
3. Create and configure CSRF middleware
4. Add CSRF middleware to the GET form route
5. Generate token using `req.csrfToken()`
6. Embed token in form (Twig hidden input)
7. Validate token in POST submission route
8. Server prevents forged requests

This ensures attackers cannot submit forms from external websites because they do not possess the valid CSRF token stored in the user's cookie/session.
